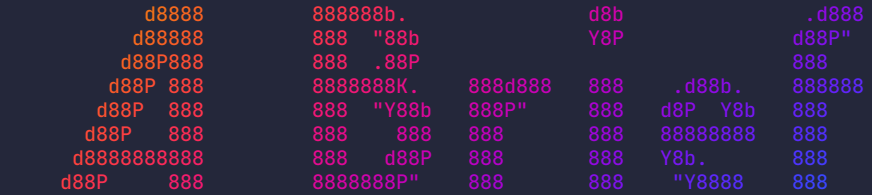
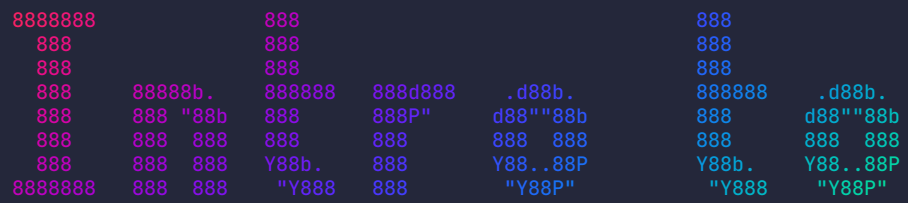
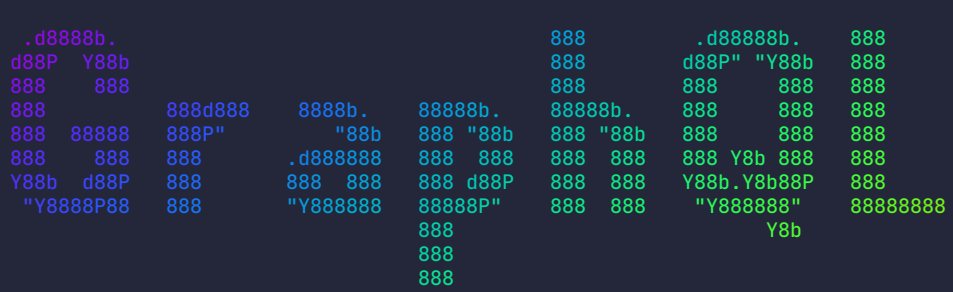

d8888 888888b. d8b .d888
d88888 888 "88b Y8P d88P"
d88P888 888 .88P 888
d88P 888 8888888K. 888d888 888 .d88b. 888888
d88P 888 888 "Y88b 888P" 888 d8P Y8b 888
d88P 888 888 888 888 88888888 888
d8888888888 888 d88P 888 888 Y8b. 888
d88P 888 8888888P" 888 888 "Y8888 888

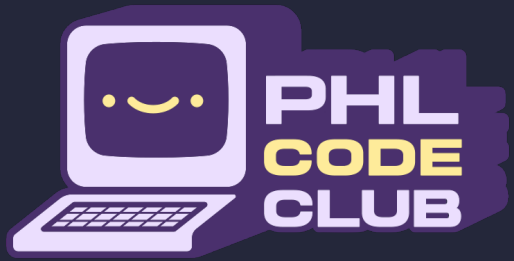

8888888 888 888
888 888 888
888 888 888
888 88888b. 888888 888d888 .d88b. 888888 .d88b.
888 888 "88b 888 888P" d88""88b 888 d88""88b
888 888 888 888 888 888 888 888 888
888 888 888 Y88b. 888 Y88..88P Y88b. Y88..88P
8888888 888 888 "Y888 888 "Y88P" "Y888 "Y88P"


.d8888b. 888 .d88888b. 888
d88P Y88b 888 d88P" "Y88b 888
888 888 888 888 888 888
888 888d888 8888b. 88888b. 88888b. 888 888 888
888 88888 888P" "88b 888 "88b 888 "88b 888 888 888
888 888 888 .d888888 888 888 888 888 888 888 888
Y88b d88P 888 888 888 888 d88P 888 888 888 Y8b 888 888
"Y8888P88 888 "Y888888 88888P" 888 888 "Y888888" 88888888
888 Y8b
888
888

Introduction

Who even is this guy?

- Principal Software Engineer
- Keyboard and ergonomics enthusiast
- Die hard terminal fanboy
- Loves typesystems and statically typed languages
- Co-founder and Chief Terminal Nerd @ PHL Code Club



And here is my cat Fritzy =3

What even is this?

A couple quick ground rules about what this talk *is* and **isn't**.

Is

- A quick and dirty overview of the main components/ideas behind GraphQL
- A fun and lighthearted stepping off point for anyone who is interested in or new to GraphQL
- Some time for me to yap because I like yapping

Isn't

- A full technical deep dive into the GraphQL type system or runtime
- An authoritative list of when to use GraphQL
- Me taking a side in a holy war between REST and GraphQL (I mean that's ***so*** 2018)

Just be respectful plz. I will have space at the end for questions if you have any!

What is GraphQL?

Stolen from **GraphQL.org** (*https://graphql.org*)

> GraphQL is a query language for APIs and a runtime for fulfilling those queries with your existing data. GraphQL provides a complete and understandable description of the data in your API, gives clients the power to ask for exactly what they need and nothing more, makes it easier to evolve APIs over time, and enables powerful developer tools.

Can we break it down?

GraphQL is split into 2 main components

GraphQL: the Language

- Split into 2 subsets
  1. Schema Definition Language (SDL)
     - Used for defining the shape of your data. Think of it like declaring a schema for a table in SQL
  2. Query Language
     - Used for querying GraphQL APIs. This is dictated by and validated against the schema using the 2nd component...

GraphQL: the Runtime

- The actual engine the handles API requests
- Validates GraphQL queries against GraphQL schemas to verify that the API supports what is being asked of it
- Resolves the query based on provided resolver functions
  - Often times these return a static value or fetch data from a DB or external API. The returned value just needs to match the schema's return type

Let's get an example

Schema

```
type Project {
  name: String
  tagline: String
  contributors: [User]
}
```

Here we declare a `Project` type. This has 3 fields:

- `name`
  - This is a string type
- `tagline`
  - Also a string
- `contributors`
  - This one is a list of `User` types
  - GraphQL lets you nest your types to allow for easy querying of related data by clients

Query

```
{
  project(name: "GraphQL") {
    tagline
  }
}
```

Now we are going to send a GraphQL query to our server. Here we describe the shape of data we want to return. Notice how we are able to ask for specifically what we want, which reduces under and over fetching In this query we are getting the tagline of the `Project` with the name `GraphQL`

Result

```
{
  "project": {
    "tagline": "A query
language for APIs"
  }
}
```

Finally we get back a result. This result will be JSON that is the same format as our query, which makes extracting nested data very easy!

Schema Definition

GraphQL Schemas are built from a number of useful types. These range from scalar types (Think strings and numbers), to complex object types (Like a user or a product). However they all start with the main schema definition*

Schema

- This is the representation of the root level operations that can be handled by a GraphQL server. Sort of like rules about what a server can and can't do

Query

- Used for requesting data from the API
- Similar to how the GET HTTP method is used by REST APIs
- Examples:
  - Requesting user data
  - Requesting a list of products

Mutation

- Used for makings stateful changes on the service
- Similar to how POST/PUT/PATCH
- Examples:
  - Logging a user in
  - Submitting an order

Subscription

- Used for requesting data similar to Query, but also updates whenever this data by REST AP changes
- This is generally implemented using either WebSockets or Server Sent Events
- Examples:
  - Watching order status
  - Live package tracking

*Technically you don't *need* an explicit schema definition, but you can ask me about that later ;)

Type Definitions

There are a handful of helpful type system definitions you can use to express your data. They include:

- Scalar
  - Primitive values like strings, numbers, booleans, etc
- Object
  - Sets of key/value pairs that can be used to represent complex data
- Enum
  - Fixed sets of values that are known at runtime
- Input
  - Like Objects but are only used for inputs to resolvers (More on that later)
- List
  - A list of zero or more of one of the other types
- Interface
  - Common field sets that an Object and implement
- Union
  - A union over a set of types (Basically an or type)
- Arguments
  - Used to specify additional data that will be used when calling GraphQL resolvers

Scalar Type

- keyword: scalar
- Allows you to declare custom scalars (Sort of like primitive values)
- The built in scalars for GraphQL are:
  - String
  - Int
  - Float
  - Bool
  - ID
    - This one is a little unique and beyond the scope of this talk

```
1 scalar Url
```

Object Type

- keyword: `type`
- Used to express complex types
- They consist of a name and a set of fields:
  - These fields are key-values
    - The key is the field name
    - The value is the field type
  - Field types can be scalars, object types, interfaces, unions, enums or lists
  - There is also a special syntax for denoting whether or not a field is optional (or `nullable` in GraphQL terms)

```graphql
type CPU {
  id: ID!
  name: String!
  baseClock: Float!
  turboClock: Float
  coreCount: Int!
  cache: Int!
  socket: Socket!
  tdp: Float!
  canOverclock: Bool!
}

type Socket {
  name: String!
  maxPowerRating: Float!
}
```

Enum Type

- keyword: enum
- Enumerations are like lists of possible values. Very handy when you have a predefined set of options that you want to enforce at the type level

```
enum BusinessType {
  Retail
  Hospitality
  Service
}
```

Interface Type

- keyword: interface
- Declare a common set of fields that an object type can implement. This allows you to verify that a type has a particular basic shape that can be extended to allow of additional information

```
interface NamedEntity {
  id: ID!
  name: String!
}

type Business implements NamedEntity {
  id: ID!
  name: String!
  type: BusinessType! # The enum we declared last slide
}

type Person implements NamedEntity {
  id: ID!
  name: String!
  age: Int
  friends: [Person!]!
  photo: Url
}
```

Union Type

- keyword: union
- Describes a union of types, basically an *or*. This is important for when you can return multiple possible types, but they don't all implement a common interface

```
type User {
  name: String!
  photo: Url
}

type Post {
  body: String!
  author: User!
}

union SearchResult = User | Post
```

Input Type

- keyword: input
- Declare the shape of an input object. This will make more sense when we get into querying data

```
input CreatePersonInput {
    name: String!
    age: Int
    friends: [ID!]
    photo: Url
}
```

Arguments

- Parens that come after a field in an Object type or Schema declaration
- Used to pass additional data to resolvers at runtime.

```
type UserWithPagedPosts {
  user: User!
  posts(skip: Int, limit: Int): [Post!]!
}
```

GraphQL Operations

The other side of GraphQL is describing the data you want to query using operations and fragments:

Fragments

- These let you create reusable snippets of fields on a particular type. These can either be explicitely created using the fragment keyword, or inline when working with interfaces and unions

Operations

- These are how you tell a GraphQL service what data you want. They can be one of the three root types:
  - query
  - mutation
  - subscription
- You can only operate on what the schema supports
- If you try to perform a mutation on a service that doesn't support them, you will get an error!
- These are going to be a set of fields that you want to query, including any arguments, called a selection set

Example GraphQL Operations

If we use an example schema:

```
type Query {
    person(name: String!): Person
}

type Mutation {
    create(input: CreatePersonInput):
Person
}

scalar Url

type Person {
  id: ID!
  name: String!
  age: Int
  friends: [Person!]!
  photo: Url
}

input CreatePersonInput {
    name: String!
    age: Int
    friends: [String!]
    photo: Url

}
```

We can do some operations:

```
fragment NameAndPhoto on Person {
    name
    photo
}
query GetPerson {
    person(name: "Graham Vasquez") {
        ...NameAndPhoto
        friends {
            ...NameAndPhoto
        }
    }
}
mutation CreatePerson {
    createPerson(input: {
        name: "Fritzy",
        age: 4,
        friends: ["Graham Vasquez"],
        photo:
"https://gvasquez.dev/cat.png"
    }) {
        ...NameAndPhoto
        age
        friends {
            name
        }
    }
}
```

Introspection

One really great feature about GraphQL is that it is **"Self Documenting"**. That is a bit of an overstatement since you can only encode so much information into the typesystem.

GraphQL has a feature called Introspection. It allows you to query information about the schema using the same GraphQL syntax we use for querying data. This feature enables awesome features such as client code generation and editor auto-complete. The specifics of how this works is out of scope for this talk, but feel free to ask questions during Q&A!

In this case Werner is wrong... Introspection is ***AWESOME***!

Examples plz

Let's take a look at an example GraphQL service!

Countries API (*https://countries.trevorblades.com/graphql*)

But y tho?

Let's compare GraphQL to some other web api technologies:

Technology	Maturity	Type Safety	Support*	Not XML
REST	✅	❌	✅	✅
SOAP	✅	✅	✅	❌
gRPC	✅	✅	❌	✅
TRPC	❌	✅	❌	✅
GraphQL	✅	✅	✅	✅

* This mostly talks about cross client support (Web, mobile, other servers, etc)

However, this flexability isn't free. GraphQL has a decently large runtime that makes all this possible. In performance critical situations, GraphQL might not be the ideal candidate when every ms counts.

However, when you have to work with multiple client teams and you can spare some clock cycles and network bandwidth, GraphQL can make handling the data layer of your API a breeze.

Q&A

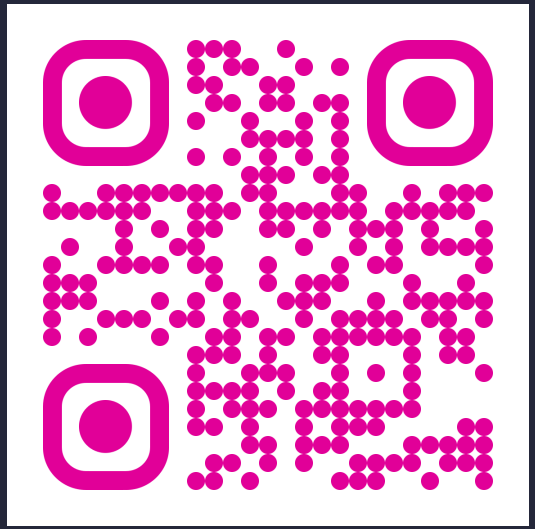

WATCH OUT!
DODC AGENTS
TARGET
COLLECT!
HOODIES!
ENTER
WATCH OUT!
Any questions?
Disney+

Additional Resources

Here are some great additional resources if you want to learn more

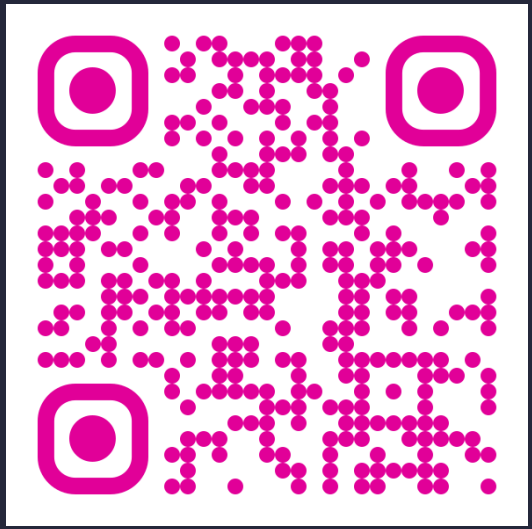
GraphQL.org: Learn (*https://graphql.org/learn*)

- Official GraphQL learning material



GraphQL.org: Resource Hub (*https://graphql.org/resources*)

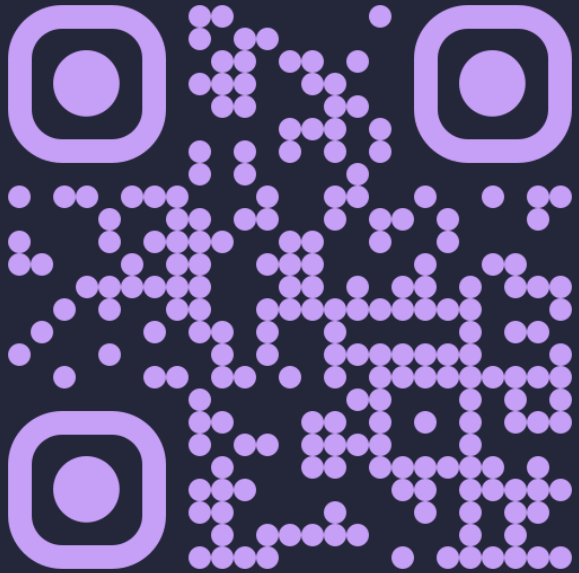
- Additional resources split by type



Contact

Feel free to reach out to me

Website (*https://graham@gvasquez.dev/*)



Fin